

NEXTGEN PYTHON INTERNSHIP MX · Q2 2026 · NOSQL WORKSHOP

Workshop NoSQL DB

MongoDB

Case study: how Forbes rebuilt their CMS on MongoDB

Follow-along session

Hands-on, all levels welcome

A DIFFERENT MENTAL MODEL

Same skills, a different mental model

POSTGRESQL (PREVIOUS WORKSHOP)

- Rows in fixed-column **tables**
- Schema changes need a **migration**
- Relationships resolved with **JOINS**
- One shape per table, always

MONGODB (THIS WORKSHOP)

- **Documents** in flexible collections
- New shapes need **zero migration**
- Relationships: **embed** or **reference**
- Different shapes, same collection

Flexible by design, so you can ship new shapes fast.

Why a magazine rewrote its entire CMS

- Forbes needed to publish **articles, videos, slideshows, and sponsored posts** — each shaped completely differently
- A relational schema meant either one giant table full of NULLs, or a new migration for every new content type
- They rebuilt the CMS on MongoDB and shipped it in **2 months**

“MongoDB’s document model meant developers could build new features quickly, easily incorporate changes, and better handle a growing diversity of data types.”

TODAY, WE REBUILD IT

Curriculum roadmap

#	FILE	TOPIC	THE FORBES NEED IT ANSWERS
01	01_crud.js	CRUD basics	Get comfortable reading/writing documents first
02	02_schema_design.js	Schema design	4 content types, 1 collection, no migrations — the actual reason Forbes chose Mongo
03	03_aggregation.js	Aggregation	“Real-time insight into social sharing” — Forbes’ own words
04	04_indexes.js	Indexes	Serving ~121M monthly users without a collection scan

Pitching MongoDB to a real project

- Not "just store JSON" — a bet that your data shapes are still evolving, and you want to ship without a migration cycle every time
- ✓ **Content shapes that vary by type** or change often — articles vs. videos, product categories with different attributes each
- ✓ **Nested, hierarchical data** that's naturally read together — an order and its line items, a user and their preferences
- ✓ **Real aggregation/analytics needs** on that data, not just simple lookups
- ✓ **A team iterating fast** — shipping new fields weekly, that can't wait on a schema migration

Mostly rigid, well-known entities with heavy cross-table reporting? Plain SQL is still the simpler choice.

Who else runs on this exact pattern

- **Forbes** — 4 content types in 1 collection, no migrations, shipped in 2 months (today's whole case study)
- **Cisco** — migrated a \$40B/year eCommerce platform off a relational database: **5x lower latency, 100x better availability**
- **Toyota Connected** — vehicle telematics platform on MongoDB Atlas, targeting 99.99%+ uptime
- **eBay** — product catalog & search metadata across 1.4 billion active listings, 190 markets

Sources: MongoDB customer case studies (mongodb.com/solutions/customer-case-studies) — Cisco, Toyota Connected, eBay

COMMON MISCONCEPTIONS

What MongoDB is NOT

- **Not schema-less.** It's schema-*flexible* — you can (and often should) enforce structure with `$jsonSchema` validation, like Exercise 2's Part C. "Flexible" means the rules are optional and per-collection, not that there are no rules.
- **Not transaction-free.** Unlike Cassandra (this program's other NoSQL workshop), MongoDB supports real multi-document ACID transactions — since version 4.0 for a single replica set, and 4.2 across sharded clusters. The honest tradeoff: reaching for them constantly is usually a sign your schema should be embedding more, not less — transactions carry a real performance cost.

How MongoDB compares

SYSTEM	SCHEMA MODEL	TRANSACTIONS	BEST FOR
MongoDB	Flexible documents, schema optional	Multi-doc ACID (since v4.0 / v4.2)	Evolving content shapes, nested data, fast iteration
PostgreSQL + JSONB	Relational, JSON columns bolted on	Full ACID	Adding flexible fields to an otherwise relational app
DynamoDB	Key-value, fully managed	Limited, constrained multi-item	Massive scale, zero operations, simple key access
Cassandra	Wide-column, no leader	No cross-partition transactions	Extreme write throughput, no single point of failure

CRUD basics

- Read: `find()` / `findOne()` , filters, sort, limit, count
- Write: `insertOne` / `insertMany` — updates use operators, not plain assignment
- No JOIN: resolve `author_id` yourself

JS

```
db.articles.updateOne(
  { slug: 'x' },
  { $push: { comments: {...} } }
);
// operators, not plain assignment
```

- Goal isn't the syntax — it's getting comfortable before Exercise 2's real payoff

Schema design: embed vs. reference

- Same collection, four shapes — **zero migrations**
- **Embed** comments vs. **reference** authors (one author, thousands of articles)
- `slug` is the one field every shape keeps fixed

```
JS
db.articles.aggregate([
  { $lookup: {
    from: 'authors',
    localField: 'author_id',
    foreignField: '_id',
    as: 'author' } },
  { $unwind: '$author' }
]);
// the closest thing to a JOIN
```

- `$jsonSchema` validation: MongoDB is schema-**flexible**, not schema-less

Aggregation pipelines

- `find()` can't compute "average shares per tag" — needs `$group`
 - A pipeline is an **array of stages** — order matters, each stage feeds the next
 - This is Forbes' own words: **"real-time insight into social sharing"**
-
- Comment-level analysis with `$unwind` and `$size`

JS

```
db.articles.aggregate([
  { $unwind: '$tags' },
  { $group: {
    _id: '$tags',
    avg_shares: { $avg: '$stats.shares' }
  } }
]);
// _id here = grouping key, NOT doc _id
```

Indexes

- `explain()` first — see **COLLSCAN** become **IXSCAN**
- Unique index on `slug` — two articles can never collide on a URL
- Compound index `(type, published_at)` matches the real query

JS

```
db.articles.createIndex(  
  { slug: 1 }, { unique: true }  
);  
  
db.articles.find({ slug: 'x' })  
  .explain('executionStats');  
// stage: "COLLSCAN" -> "IXSCAN"
```

- Text index for on-site search, with a relevance score

BEFORE WE START

Get the stack running

- `git clone github.com/agent3dev/workshop-nosql-stage2-mongodb`
- `docker compose up -d`
- Visual UI: **mongo-express** at `localhost:8081`
- Or terminal: `docker exec -it forbes_mongo mongosh ...`

Full setup steps are in the repo README